Graph Reasoning on Large Language Models for Logistics Networks

Dylan D'Silva
dept. of Mechanical and Aerospace Engineering
University of Central Florida

Raymond Correa
dept. of Modeling and Simulation
University of Central Florida

***Abstract*—Containerized cargo logistics relies on complex routing decisions across a network of ports, hubs, and shipping routes. Network analyses typically use purpose-built graph algorithms. Advances in Large Language Models raise the possibility of a more accessible interface for such reasoning, using conversational queries, but it remains unclear how reliably LLMs can perform computations when provided with text representations of the graph. We construct a directed, weighted graph representing a shipping network in which nodes are ports and distribution hubs, and edges represent routes with travel time, monetary cost, and risk attributes. We serialize the graph into edge-list, adjacency-list, and incidence encodings. These encodings are supplied to an LLM along with queries of path existence and optimal pathing. LLM responses are parsed and compared against reference answers computed using graph algorithms, with performance reported as exact-match accuracy, path equivalence, and cost ratios.**

I. Introduction

A. Background and Motivation

Containerized cargo shipping involves complex logistics networks with routing decisions that are difficult to analyze using conventional tools. Logistics operators and routers must routinely determine cheap and feasible routes. Nodes of logistics networks can possess a variety of meanings, from ports to rail terminals to distribution centers. Shipping routes, the edges connecting these nodes, are rarely static. They vary with weather and seasons, geopolitical interactions, and port congestion. Determining routes today requires purpose-built software.

Advances in Large Language Models (LLMs) raise the possibility of a more accessible interface for working with logistics graphs, with the potential development of Software as a Service (SaaS). A user could describe the network to the LLM and ask a question using natural language, receiving a natural language answer. This could make logistics tools easier to use and implement. The framework fits naturally into a SaaS tool in which dispatchers, route planners, and analysts could query the LLM about the network conversationally rather than through a specialized interface.

It is still uncertain whether current LLMs can reliably perform the underlying graph reasoning. Prior work has shown that LLMs can be capable of handling small graph problems, but performance degrades as graph size or density increases. Understanding where the degradation begins in these networks is a prerequisite for deploying LLMs in logistics settings, for which an incorrect answer would have a real and significant cost.

B. Related Works

We draw from two concepts in this study: benchmarks evaluating LLM graph reasoning, and studies on how graph-to-text encoding affects LLM performance. Several recent benchmarks evaluate the ability of LLMs to perform reasoning over graphs and graph-structured data. The NLGraph benchmark [1] reports exact-match accuracy across eight graph reasoning tasks, including connectivity (true/false correctness) and shortest path (solution cost matched against truth). NLGraph finds LLMs perform reasonably on simple tasks, such as connectivity, but degrade sharply when introduced to more complex reasoning tasks such as maximum flow. GraphArena [2] extends this by classifying LLM outputs into correct, suboptimal, hallucinatory, and missing categories to provide a better view of failure modes. GraphWiz [3] discusses that stronger general-purpose LLMs achieve roughly 45% average accuracy across a set of graph computations including connectivity and path optimization.

These works suggest LLMs have meaningful but limited capacities for graph reasoning with performance degrading with increases in graph size and density as well as task complexity. A separate line of work examines how the textual representation of a graph affects LLM performance. The survey in [4] catalogs common serializations including edge-list, adjacency-list, and incidence encoding. [5] shows the selection of serialization can cause a 61% change in performance for a given graph, indicating encoding is a significant experimental variable.

II. Problem Statement

We study how LLMs handle reasoning over logistics graphical networks represented as text. Given a directed, weighted graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, representing a logistics network, and a natural-language query, q, about $\mathcal{G}$, can an LLM produce an answer that matches the answer obtained by running an exact graph algorithm on $\mathcal{G}$.

The networked system being studied is a containerized-cargo logistics network. Nodes in $\mathcal{V}$ represent ports and connection points such as shipment hubs. Directed edges $\mathcal{E}$ represent routes between these locations. Each edge has attributes which determine its suitability for a given query,

including travel time, monetary cost, and weather risk. We serialize the graph into text and include it in the prompt along with the query. We parse the LLM response and compare the solution to a ground truth answer computed by NetworkX on $\mathcal{G}$.

Within this framing, we explore how the method of graph textual encoding (edge-list, adjacency-list, and incidence) affects LLM accuracy on logistics queries. This study compares which methods work best for LLM graph reasoning, together with an analysis of the query types on which encoding is most and least effective.

III. Methodology

There are three steps to our approach: construction of the logistics graph, conversion of the graph into text for use in prompts and queries, and comparison of LLM answers to ground truth. Ground truth solutions are computed using exact algorithms. Our queries and evaluations are inspired by the NLGraph metric, which uses several different query categories for analyzing LLM performance.

A. Graph Construction

The logistics network is a directed, weighted graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$. $\mathcal{V}$ is the set of nodes (representing ports), $\mathcal{E}$ is the set of directed edges (routes between locations), and $\mathcal{A} : \mathcal{E} \to \mathbb{R}^+ \times \mathbb{R}^+ \times \{\text{low, medium, high}\}$ represents edge attributes describing transport time, cost, and risk.

We construct a synthetic graph of 25 nodes and 50 edges representing major North American shipping and air freight hubs. Edges represent routes between hubs. Route attributes were assigned to reflect plausible transport conditions, with shorter routes generally exhibiting lower cost and travel time. The graph is shown in Figure 1 with edges colored by weather-risk categorization.

A synthetic graph was used in place of a public dataset because publicly available logistics datasets either lack route level attributes or are too large to encode within a single LLM prompt. Constructing a graph by hand allows us to control graph size, density, and attributes while preserving the structure of a realistic logistics network.

B. Graph-to-Text Serialization

Three representations of the graph are tested: edge-list, adjacency-list, and incidence encoding. Each represents the same graph with the same attributes. Only their textual forms differ. These serializations are defined below with examples.

- Edge-list encoding. One edge per line of the form $v_i \to v_j$ followed by edge attributes:

```
JFK -> LAX [time=5d, cost=$1200, weather=
    low]
```

- Adjacency-list encoding. Each line consists of a node, its outgoing neighbors, and their respective edge attributes:

```
JFK: LAX (5d, $1200, low), BOS (1d, $400,
     low), ATL (2d, $700, medium)
```

- Incidence encoding. Similar to adjacency-list encoding but uses natural language:

```
From JFK, routes are available to LAX (5
    days, $1200, low weather risk), BOS
    (1 day, $400, low weather risk), and
    ATL (2 days, $700, medium weather
    risk).
```

We do not vary the underlying graph, the query set, or the model between conditions; encoding is the only experimental variable. Any difference in LLM accuracy can therefore be attributed solely to the encoding method rather than to differences in the graph itself.

C. Query Set

Three query types are defined, representing the questions a logistics user and operator would be interested in. These questions stem from determining the existence of a path between two nodes, the fastest shipping route available, and the cheapest shipping route available. Connectivity checks if a path exists between port $\mathbf{X}$ and port $\mathbf{Y}$. The shortest route minimizes

$$P^* = \arg\min_P \sum_{e \in P} t_e \quad \text{s.t.} \quad P \subset \mathcal{E}$$

where t_e is the time attribute of a given edge. The cheapest route minimizes

$$P^* = \arg\min_P \sum_{e \in P} c_e \quad \text{s.t.} \quad P \subset \mathcal{E}$$

where c_e is the cost attribute of a given edge. These optimizations have the additional constraints that $\mathbf{X}$ is the source of P and $\mathbf{Y}$ is the target of P.

Each query category uses a fixed set of $(\mathbf{X}, \mathbf{Y})$ pairs sampled from the graph. These are sampled uniformly at random from $\mathcal{V} \times \mathcal{V}$, excluding samples where a node pairs with itself. The same node pairs are used for all three encodings. For shortest and cheapest routes, we also require that the nodes are connected.

Prompts are passed to Claude Sonnet 4.5 through its public API. Each prompt to the LLM has three components: a fixed message describing the task and response formatting, the serialized and text encoded graph, and the query.

D. Ground Truth and Evaluation

For each query, we find the ground truth answer using NetworkX. Metrics for evaluating LLM performance are inspired by NLGraph [1]. Connectivity is verified using `has_path`. Shortest paths are computed using Dijkstra's algorithm with `shortest_path`.

For path finding queries, we report two notions of correctness. Optimal accuracy reports the fraction of queries for which the LLM's returned path achieves the optimal total cost. Any path that ties with the solution from Dijkstra's algorithm is accepted. Exact accuracy reports the fraction of queries for which the LLM's solution matches the path returned by Dijkstra exactly. These two metrics match when the optimal

path is unique. When the optimal path is not unique, the two values may diverge if the LLM selects a different path.

We also compute a cost ratio. This is the ratio of the LLM's solution cost to the optimal cost, applied to both the fastest path and lowest cost path queries. A ratio of 1 indicates an optimal solution, and a ratio greater than 1 indicates a suboptimal path. The cost ratio provides a non-dimensional reference with which we can compare LLM solutions.

IV. Results

Figure 1 shows the synthetic graph used for analysis. The entire network contained 25 nodes and 50 edges. A set of 32 node pairs are selected from this graph to analyze the accuracy of the LLM in determining node connectivity. Of those 32 node pairs, 19 pairs had valid and existing paths between them. The original 32 pairs were used for the connectivity performance discussion. Only the 19 with valid paths were considered for the optimal pathing problems. The number of queries was primarily limited by the cost of querying the Claude API. Across all results, 210 queries were made.

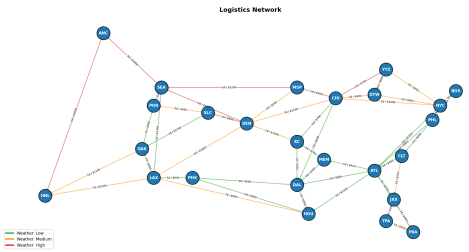

Fig. 1. Synthetic Logistics Network

A. Node Connectivity

Determining the connectivity between two nodes is one of the simplest graph reasoning tasks for the LLM. Only 19 of the 32 randomly sampled node pairs had valid paths. Those 19 pairs are used in the optimal pathing queries below. Figure 2 shows the exact-match accuracy against NetworkX's `has_path` for each encoding.

The encoding methods showed a tight grouping, with all three achieving high accuracy. Incidence encoding led at 97%, followed by edge-list at 94%, and adjacency-list at 91%. Keeping in mind the sample size of 32 node pairs, the difference of 3% and 6% corresponds to a disagreement of one and two answers respectively. Because connectivity is the simplest reasoning task, we can expect LLM performance to generally be at or below this accuracy for the remaining queries.

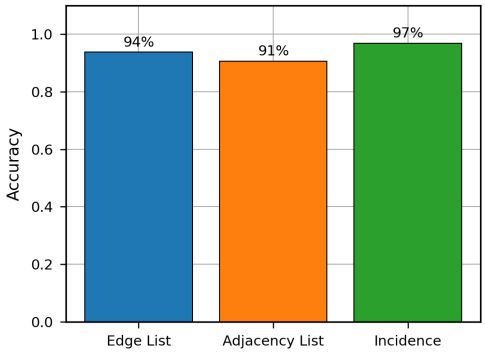

Fig. 2. LLM Node Connectivity Accuracy

B. Fastest Path

Using edge-list encoding, the LLM was able to completely recover the solutions obtained using exact algorithms. Incidence encoding showed a slight degradation in performance, finding optimal paths in 95% of the test cases. With a sample size of 19 node pairs, the 5% error (rounded) corresponds to a single misclassified result. Under adjacency-list encoding, there was a significant drop in performance to 79% accuracy. Figure 3 shows the accuracy of the LLM across the test cases for each serialization method. The blue bars represent how frequently the LLM returned a path achieving minimal travel time. The orange bars represents how often the optimizing path found by the LLM exactly matches the path found by the search algorithm.

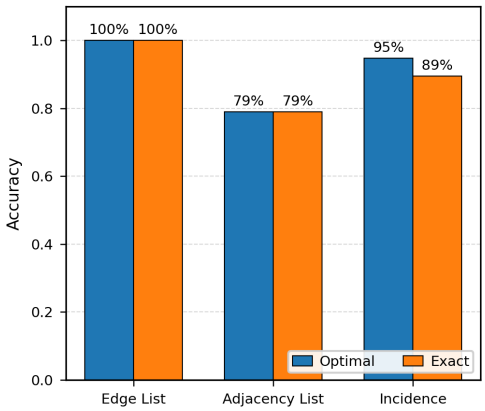

Fig. 3. LLM Fastest Path Accuracy

For adjacency-list encoding, when the LLM found a truly optimal solution, its solution always agreed with the search algorithm. When queried with incidence encoding, the LLM found a solution with the minimum time, but the path was not the same as the path found by NetworkX's `shortest_path()`. Instead, the LLM found an alternative path. The existence of alternative paths is a consequence of how edge travel time attributes were described. The inclusion of fractional days may have altered the number of optimal and alternative paths found by the LLM.

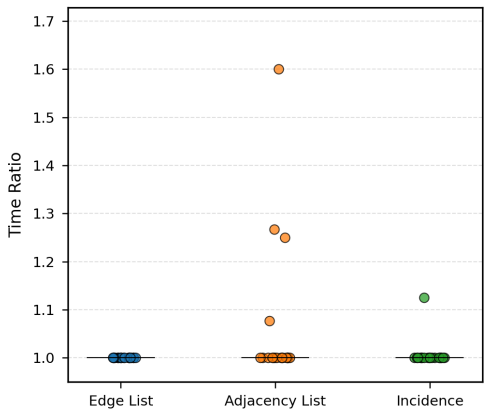

Fig. 4. LLM Solution Transport Time Ratio

Figure 4 shows the time ratio of the LLM solution. Edge-list encoded queries achieved ratios of 1 across the board. This is consistent with their 100% accuracy in Figure 3. Incidence encoding produced one suboptimal query, while adjacency-list encoding produced three notable outliers, including one solution with a ratio of 1.6, meaning the LLM path took 60% longer than the optimal route.

C. Cheapest Path

Figure 5 shows the accuracy of the LLM in determining a minimum cost path between two nodes. Interestingly, there is a difference in LLM performance for determining the fastest path compared to the lowest cost path. Compared to the minimal time queries, the LLM is able to find more alternative optimal paths. The worse performance and lower "Exact" metric suggests a systematic difference in how the LLM selects paths compared to the fastest path queries and Dijkstra's algorithm.

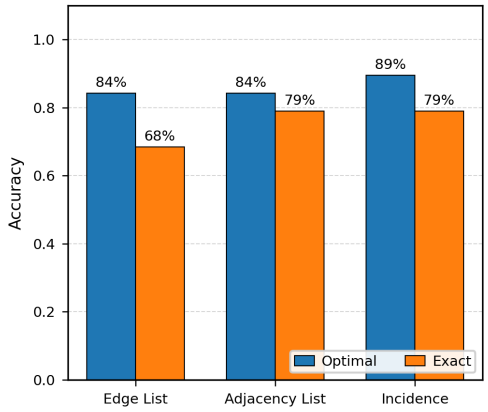

Fig. 5. LLM Cheapest Path Accuracy

From Figure 6, the model usually finds paths at the optimal cost, regardless of whether it is the same path as what Dijkstra's algorithm would find. For edge-list encoded queries, even sub-optimal paths were near-optimal with one notable outlier. The model provided a solution path with a cost more than double the optimal path. Similarly, but to a lesser extreme, the incidence serialization results show a significant outlier with a cost ratio of about 1.7. That means the model determined a path for which the transport cost would be 70% above optimal.

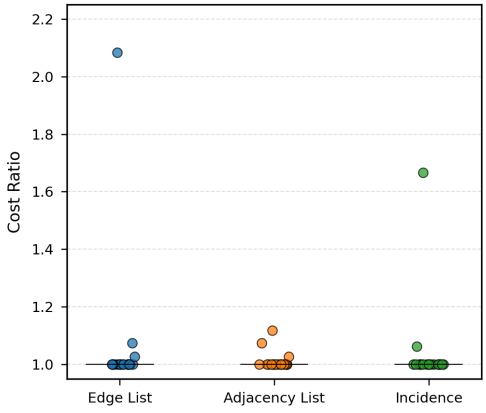

Fig. 6. Cost Ratio

V. Conclusion

Our results show that LLM accuracy on graph reasoning is highly dependent on both the query type and the choice of textual encoding. Across all three query types, edge-list and incidence encoding outperform adjacency-list encoding. Edge-list encoding was best on fastest-path, lowest-travel time queries, and incidence encoding showed a slight lead on connectivity and lowest-cost route queries. The gap between Optimal and Exact metrics in the cheapest path accuracy suggests that lowest cost queries produce more paths for which optimality is not unique, and that the LLM may select non-deterministically from among them. Cost ratios remain close to 1 for the most part, showing rare but significant outliers. One such solution is more than double the optimal value, which serves as a concern for any tool deployed in a logistics setting. These occasional large errors have the potential for costly consequences in both time and monetary cost.

Taken together, these results suggest LLMs can serve as a usable interface for queries on small logistics networks, but the failure modes observed make them unsuitable as a single source of truth in deployed tools without additional development. Compared to broader benchmarks reporting low accuracy on graph reasoning [1]–[3], our metrics are higher. This is likely because our graph is a small network with only 25 nodes and 50 edges. Our graph is likely both smaller and more connected than the graphs used for those benchmarks. We also provide a narrower set of query types, and we use a recently released model. Real logistics networks would be significantly larger, and the accuracy loss we observe at this scale suggests that queries with a serialized graph will not scale to larger networks.

Several limitations bound the applicability of our results. We tested a single model on a synthetic graph using a handful

of queries. A single misclassifications corresponds to a 3-5% performance drop, so small differences between encoding methods should be considered carefully. We also did not vary prompt structure or apply reasoning techniques known to improve performance. Several approaches could improve the LLM performance on this task. Among them, chain-of-thought prompting [6], [7] is the most directly applicable, as path finding requires multi-step reasoning for which the model is not currently asked to consider or make explicit. Beyond prompting changes, augmenting the graph encoding with node-level features such as centrality or commerce throughput could bias the model toward commonly used hubs, while randomizing or sorting the order in which edges are presented could mitigate any positional bias in the input. Each of these is ultimately a matter of prompt and context engineering and serves as a promising path toward scaling this approach towards real logistics.

REFERENCES

[1] H. Wang, S. Feng, T. He, Z. Tan, X. Han, and Y. Tsvetkov, "Can language models solve graph problems in natural language?" in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[2] J. Tang, Q. Zhang, Y. Li, N. Chen, and J. Li, "Grapharena: Evaluating and exploring large language models on graph computation," in *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, 2025, pp. 97 329–97 356.

[3] N. Chen, Y. Li, J. Tang, and J. Li, "Graphwiz: An instruction-following language model for graph computational problems," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024)*, Aug. 2024, pp. 353–364.

[4] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, and J. Han, "Large language models on graphs: A comprehensive survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 12, pp. 8622–8642, 2024.

[5] B. Fatemi, J. Halcrow, and B. Perozzi, "Talk like a graph: Encoding graphs for large language models," in *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 24 824–24 837.

[7] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023.